JAVA SDE-2 INTERVIEW PREPARATION SERIES

STUDY STEP 08 OF 18

Hashing: Maps, Sets, Frequency, and Prefix State

Equality Contracts, Frequency Maps, Deduplication, and Expected Complexity

FOUNDING AUTHOR

Vinay Reddy Kalluri

EDITOR-IN-CHIEF | CHIEF AUDITOR

Understand equality and hashing deeply enough to use maps and sets for frequency, membership, prefix-state, caching, and custom-key interview problems.

HASHING | MAPS | SETS | FREQUENCY | PREFIX STATE | EQUALITY

JAVA 21 | INTERVIEW CORE | SDE-2 FOLLOW-UPS | PRINTABLE EDITION

Series edition - Release 2026-07-25

Open educational interview-preparation edition

Start Here - Your Route Through This Volume

> **60-second entry rule:** If two or more recognition signals below expose a gap, begin with Chapter 1. If the prerequisites are weak, step back to `Study Step 07 — Strings and String Patterns`. If you can already explain every outcome without notes, jump to Part II and prove it through the practice ladder.

Hashing is both a DSA pattern and a Java object-contract problem. This volume connects the two so expected O(1) claims remain defensible.

Readiness map

Decision	Evidence
START HERE	Fast membership or deduplication is required; A value must map to count, index, group, or state; Prefix states repeat; A custom key needs stable equality and hashing.
PREPARE FIRST	Study Steps 01-07; Equality and array/string basics.
MOVE ON WHEN	Design correct equals and hashCode contracts; Use frequency, membership, grouping, and prefix-state patterns; Explain collisions, resizing, load factor, and expected complexity; Avoid mutable-key and cardinality failures.

Choose the route that fits today

1. **Learning:** read in order, type the representative Java examples, and complete each dry run.
2. **Revision or rehearsal:** start at the first decision map, invariant, or failure mode you cannot explain; if none expose a gap, go directly to Part II and prove readiness without notes.

Study Path Position

07 - Strings and String Patterns	YOU ARE HERE **08 - Hashing and Prefix State**	09 - Recursion and Backtracking

Contents

This contents page covers only the current focused PDF. Section-level navigation is available through PDF bookmarks.

CONTENTS NAVIGATION

Part I - Core Learning

SDE-2 CORE CHAPTER

Chapter 1 - Equality and Hashing Contracts

WHY IT WORKS | First-principles model

Every object has identity. Reference `==` asks whether two reference values designate the same object, with null handled as an ordinary reference value. `Object.equals` initially implements identity equality, but classes can override it to define logical value equality.

Hashing compresses equality-relevant state into an integer used to choose candidate buckets. A hash is not a unique identifier. Equal objects must have equal hash codes; unequal objects may collide. Hash-based collections use both hash and equality.

Immutability means an object's observable state cannot change after construction. It requires control of the entire reachable representation, not only final top-level fields. A record is a transparent nominal product of its components with generated accessors and value-oriented members. Its component fields are final, but referenced objects may be mutable.

> **Specification boundary:** Java specifies the `Object.equals` and `hashCode` contracts and record member semantics. It does not specify a `HashMap` bucket layout here, a stable hash code across JVM runs, or that `Object.hashCode` is a memory address.

Core terminology

- **Identity equality:** same runtime object, tested with reference `==`.
- **Logical equality:** class-defined equivalence, tested by `equals`.
- **Domain identity:** business identity, such as an immutable account identifier.
- **Value object:** object defined by its values rather than lifecycle identity.
- **Hash collision:** unequal values have the same hash code.
- **Consistency:** repeated equality or hash calls return compatible results while relevant state is unchanged.
- **Deep immutability:** no observable reachable mutable state can change through any alias.
- **Defensive copy:** independent representation used to prevent outside mutation.
- **Record component:** declared state item that generates a field, accessor, and participation in generated members.
- **Canonical constructor:** record constructor whose parameters correspond to all components.

Detailed mechanics

The equals contract

For non-null references, a correct equivalence relation is:

- reflexive: `x.equals(x)` is true;
- symmetric: `x.equals(y)` equals `y.equals(x)`;
- transitive: if x equals y and y equals z, x equals z;
- consistent while equality-relevant information is unchanged; and
- false for `x.equals(null)`.

An implementation normally checks identity, compatible type, and relevant fields:

JAVA

```java
@Override
public boolean equals(Object other) {
    if (this == other) return true;
    if (!(other instanceof Coordinate that)) return false;
    return x == that.x && y == that.y;
}
```

`getClass()` versus `instanceof` is a design choice. Exact-class equality is easier to keep symmetric in extensible hierarchies but means a base and subclass never compare equal. `instanceof` can support equality across implementations only if every participant shares the same semantics. Making value classes final or records avoids many subclass traps.

The hashCode contract

If `x.equals(y)`, then `x.hashCode() == y.hashCode()` must hold. The reverse is not required. The result must be consistent while equality state is unchanged. Overriding one of `equals` and `hashCode` without the other is almost always a defect.

JAVA

```java
@Override
public int hashCode() {
    int result = Integer.hashCode(x);
    result = 31 * result + Integer.hashCode(y);
    return result;
}
```

`Objects.hash(x, y)` is concise but uses varargs and boxing, which can matter on a hot path. Hand-written accumulation or generated code avoids that allocation. Quality matters because clustered hashes degrade hash-table performance, but correctness still relies on equality.

Never base logical hash solely on a mutable field if objects can be keys. If a key changes after insertion, lookup uses its new hash or equality state while the entry remains located according to its old state.

Equality across common types

Arrays retain identity `equals`; use `Arrays.equals` or `deepEquals`. `BigDecimal.equals` considers value and scale, so `1.0` is not equal to `1.00`; `compareTo` considers them numerically equal. This makes `BigDecimal` behavior differ between hash-based and sorted collections if the chosen semantics are not normalized.

Enums use identity safely because each declared constant is a canonical instance in its defining class loader context, and `Enum.equals` is final. Collections define content equality, usually including iteration order for lists and membership for sets.

Entity equality is difficult when database-generated IDs begin as null and proxies introduce subclasses. Do not mechanically include every mutable field. Choose a stable business key, controlled persistent identity policy, or explicit reference identity based on lifecycle requirements.

TRACE IT | Worked Java example

This record normalizes permission spelling and takes an immutable snapshot, producing stable equality and hashing.

JAVA

```java
import java.util.Set;
import java.util.TreeSet;

public record PermissionSet(String principalId, Set<String> permissions) {
    public PermissionSet {
        if (principalId == null || principalId.isBlank()) {
            throw new IllegalArgumentException("invalid principalId");
        }
        if (permissions == null) {
            throw new IllegalArgumentException("permissions is null");
        }
        TreeSet<String> normalized = new TreeSet<>();
        for (String permission : permissions) {
            if (permission == null || permission.isBlank()) {
                throw new IllegalArgumentException("invalid permission");
            }
            normalized.add(permission.strip().toLowerCase(java.util.Locale.ROOT));
        }
        permissions = Set.copyOf(normalized);
    }

    public boolean allows(String permission) {
        return permissions.contains(permission.toLowerCase(java.util.Locale.ROOT));
    }

    public static void main(String[] args) {
        PermissionSet a = new PermissionSet("u1", Set.of("READ", "write"));
        PermissionSet b = new PermissionSet("u1", Set.of("write", "read"));
        System.out.println(a.equals(b)); // true
        System.out.println(a.allows("READ")); // true
    }
}
```

Because set equality is order-independent and the canonical constructor normalizes content, different input order and case lead to the same value. Whether permission identifiers should be case-insensitive is a domain decision, not a universal rule.

WATCH OUT | Edge cases and common mistakes

- `==` on references tests identity, not logical value.
- Equal objects must share a hash, but equal hashes do not imply equal objects.
- Mutable map keys can become unreachable after insertion.
- `getClass` and `instanceof` strategies can break symmetry when mixed across a hierarchy.
- Including an array field with `Objects.equals` compares array identity; use the matching `Arrays` helper.
- `BigDecimal.equals` includes scale while `compareTo` does not.
- Unmodifiable collection wrappers do not copy their backing collections.
- Final fields do not make mutable elements immutable.
- Records can contain arrays, lists, or date objects that mutate.
- A compact record constructor can reassign parameters for normalization but should not leak partially constructed state.
- Generated `toString` can leak credentials or personal data.
- Hash codes are not stable persistence keys, signatures, or security hashes.

TRY IT | Interview questions and model answers

What is the relationship between equals and hashCode?

If two objects are equal, their hash codes must be equal. Unequal objects may share a hash. Both results must stay consistent while equality state is unchanged. Therefore a class that overrides logical equality must provide a compatible hash implementation.

Why is a mutable HashMap key dangerous?

The map places it according to its hash at insertion. If equality-relevant state changes, lookup computes a different hash or comparison and may not find the existing entry. Immutability is the simplest key policy.

Are records immutable?

Their component fields are final and the record class is final, so component references cannot be reassigned after construction. The objects referenced by components can still mutate. Defensive copies are required for deep immutability.

Should entity equality include every field?

Usually not. Mutable fields make hashing unstable and two lifecycle instances may represent the same entity. Choose a stable business identity or a carefully specified persistent-identity strategy that handles transient instances and proxies.

Can a record extend a base class?

No. Every record implicitly extends `java.lang.Record` and is final. It may implement interfaces and define behavior.

TRY IT | Exercises

1. Implement a final `Money` value with currency and minor units, including contract tests.
2. Demonstrate a map lookup failure after mutating a key, then repair the key design.
3. Compare `BigDecimal("1.0")` and `BigDecimal("1.00")` in `HashSet` and `TreeSet`; explain the result.
4. Make a record with a `byte[]` component deeply immutable, including accessor behavior.
5. Construct a base/subclass equality implementation that breaks symmetry, then redesign it.
6. Decide whether a JPA entity, API DTO, and domain identifier should each be a record, with reasons.

SDE-2 CORE CHAPTER

Chapter 2 - Collection Equality, Views, and Validity

WHY IT WORKS | First-principles model

A collection is an object that owns or exposes a group of references. The Java Collections Framework separates four ideas:

1. Interfaces define behavioral contracts, such as uniqueness or positional access.
2. Implementations choose a data structure, memory layout, and performance profile.
3. Algorithms operate through interfaces, such as sorting or binary search.
4. Views expose a live projection of another object, such as a map's key set.

The principal hierarchy is:

TEXT

```
Iterable
  Collection
    List
    Set
      SortedSet
        NavigableSet
    Queue
      Deque

Map                          (not a Collection)
  SortedMap
    NavigableMap
```

`Map` is separate because it models key-value associations rather than individual elements. Its `keySet()`, `values()`, and `entrySet()` methods bridge into the `Collection` hierarchy through views.

> **Specification boundary:** Interface contracts specify observable behavior. They generally do not promise array storage, linked nodes, hashing strategy, tree shape, amortized cost, or fail-fast detection. Complexity claims belong to the documentation of a concrete implementation.

Core terminology

- **Element:** A reference stored by a collection. Collections do not contain primitive values directly; boxing creates wrapper objects when needed.
- **Structural modification:** A change that can alter iteration structure, usually adding or removing elements. Replacing a value may or may not be structural for a particular implementation.
- **Encounter order:** The order in which an iteration mechanism presents elements. Some collections define it; others do not.
- **Natural ordering:** Ordering defined by `Comparable`.
- **Optional operation:** An interface method that an implementation may reject with `UnsupportedOperationException`.
- **View:** A projection backed by another object. Changes may be visible in both directions.
- **Snapshot:** An independent representation of state at a point in time.
- **Unmodifiable:** Mutation through that reference is rejected. It does not necessarily mean the underlying data or elements are immutable.
- **Immutable:** State cannot change after construction, including through aliases permitted by the abstraction.
- **Fail-fast iterator:** An iterator that attempts to detect unsupported concurrent structural modification and throw `ConcurrentModificationException`.

Detailed mechanics

Selecting the abstraction

Use a `List` when position, encounter order, or duplicates are meaningful. Use a `Set` for membership and uniqueness. Use a `Queue` when processing order matters and elements enter and leave through queue operations. Use a `Deque` for both ends or stack behavior. Use a `Map` when a unique key identifies a value.

Program parameters and return types to the narrowest useful interface. A method that only iterates can accept `Iterable<T>` or `Collection<T>` rather than `ArrayList<T>`. Do not hide an important semantic property, however: accepting `Set<T>` communicates uniqueness, while accepting `Collection<T>` does not.

Views and aliasing

`map.keySet()`, `map.values()`, and `map.entrySet()` are normally backed views. Removing a key from the key set removes the mapping. An entry's `setValue` can update the map when supported. `list.subList(from, to)` is also a backed view. A structural change to the parent outside the sublist can make later sublist operations fail.

Wrappers from `Collections.unmodifiableList(source)` are read-only views, not copies. If another alias mutates `source`, the wrapper reflects it. By contrast, `List.copyOf(source)` returns an

unmodifiable list whose membership is not backed by later source changes. Neither operation freezes mutable element objects.

Equality and bulk operations

`List.equals` is order-sensitive. `Set.equals` compares membership independent of order. `Map.equals` compares mappings. These contracts enable equality across implementations: an `ArrayList` can equal a `LinkedList` with the same sequence.

Bulk methods include `addAll`, `removeAll`, `retainAll`, `containsAll`, `removeIf`, and `replaceAll`. Their apparent single-call form does not imply constant time. Cost depends on both receiver and argument. For example, removing every element found in an `ArrayList` argument can be quadratic when membership checks are linear. Converting the lookup side to a `HashSet` can improve the expected bound.

Null policy and element validity

Null support varies. Some general-purpose collections allow null; many queues, concurrent collections, sorted structures with natural ordering, and factory-created immutable collections reject it. A robust API validates domain values before selecting an implementation rather than relying on an incidental null policy.

WATCH OUT | Edge cases and common mistakes

- Choosing a concrete type before defining uniqueness, ordering, and access semantics.
- Returning a mutable internal list and unintentionally granting write access.
- Assuming an unmodifiable wrapper is a snapshot or that immutable membership makes elements immutable.
- Depending on `HashMap` or `HashSet` iteration order.
- Modifying a collection directly while iterating over it.
- Treating `ConcurrentModificationException` as guaranteed detection of a race.
- Forgetting that `subList` and map collection views are backed by their owner.
- Passing a collection to itself in a bulk operation whose behavior is undefined or surprising.
- Using mutable objects whose `equals`, `hashCode`, or ordering fields change while stored.
- Assuming every implementation permits null or mutation.
- Calling `size()` repeatedly on a nonstandard collection without checking whether it is cheap.
- Confusing `Collection` with `Collections`; the latter is an algorithm and wrapper utility class.

TRY IT | Exercises

1. Design a method signature for returning active feature flags in deterministic order without allowing callers to mutate membership. State every contract not captured by the type.
2. Predict the result of removing an entry through `map.entrySet().iterator().remove()` and explain why it differs from mutating the map directly during iteration.
3. Compare `Collections.unmodifiableList(source)`, `List.copyOf(source)`, and `new ArrayList<>(source)` for mutability, null handling, and aliasing.
4. Rewrite a quadratic `removeAll` workflow so membership tests use a set. Give time and space bounds.
5. Create a small program that demonstrates a `subList` backed view. Then structurally modify the parent outside the view and record the observed behavior without treating it as a portable synchronization technique.

SDE-2 CORE CHAPTER

Chapter 3 - HashMap, HashSet, and Hashing Internals

Learning objectives

By the end of this chapter, you should be able to:

- derive hash-table lookup from equality and bucket selection;
- state the `equals` and `hashCode` contract and key-stability invariant;
- explain collisions, load factor, resizing, and expected complexity;
- describe a typical OpenJDK `HashMap` without presenting it as a platform guarantee;
- use `compute`, `merge`, views, and iteration safely; and
- identify security, concurrency, memory, and API-design risks around hash tables.

WHY IT WORKS | Why this matters at SDE-2

Hash maps support indexes, caches, joins, deduplication, frequency counts, and graph representations. They are also a rich interview subject because correctness spans language-level equality, data-structure invariants, amortized analysis, and mutation discipline.

At SDE-2, "lookup is O(1)" is incomplete. The useful answer is expected `O(1)` under a suitable hash distribution and stable key behavior, with resizing and collision caveats. You should be able to diagnose a missing entry caused by a mutated key, distinguish absence from a null value, and choose an ordered, identity-based, weak-key, concurrent, or sorted alternative when semantics demand it.

WHY IT WORKS | First-principles model

A hash table transforms a key into a candidate region rather than comparing it with every stored key. Conceptually:

```text
hashCode(key) -> mixed hash -> bucket index -> inspect candidates -> equals
```

The hash narrows the search; `equals` determines logical key identity. Different keys can have the same hash and must coexist as a collision. Equal keys must be directed to the same search region.

The central invariant is:

```text
For every stored entry (k, v), lookup using a key equal to k
must search the bucket containing that entry.
```

That requires equal objects to have equal hash codes and requires fields used by equality and hashing to remain stable while the key is stored.

`HashSet<E>` uses the same membership idea without an application value. A typical implementation is backed by a hash map whose keys are set elements and whose values are a private marker.

> **Specification boundary:** `Map` specifies unique keys and mapping behavior. `HashMap` specifies null support and general performance expectations in its API documentation, but bucket array shape, hash mixing, resize thresholds, tree bins, and constants are implementation details.

Core terminology

- **Hash code:** An `int` computed for an object and used to group possible matches.
- **Collision:** Distinct, non-equal keys assigned to the same bucket.
- **Bucket/bin:** A table position containing zero or more entries.
- **Capacity:** Number of table buckets in a typical hash-table representation.
- **Load factor:** Ratio controlling how full the table may become before resizing.
- **Threshold:** Entry count that triggers resizing, often capacity times load factor.
- **Rehash/resizing:** Allocating a larger table and redistributing entries.
- **Hash flooding:** Many keys deliberately or accidentally colliding.
- **Tree bin:** A collision bucket represented by a balanced tree in some implementations.
- **Key stability:** Requirement that equality and hash behavior not change while a key is stored.
- **Canonical key:** Stable value representation used as a map key, such as a normalized immutable identifier.

Detailed mechanics

Equality and hashing contract

`Object` establishes these practical rules:

1. If `a.equals(b)` is true, `a.hashCode() == b.hashCode()` must be true.
2. Unequal objects may share a hash code.
3. Repeated hash calls during one execution should be consistent while equality-relevant state is unchanged.

A record automatically derives value-based `equals` and `hashCode` from components, making records useful keys when components are themselves stable:

JAVA

```
record TenantUser(String tenantId, String userId) {}
```

If a mutable class uses `tenantId` and `userId` for hashing, changing either after insertion can strand the entry. The object remains physically in its old bucket while lookup computes a new bucket.

Lookup, insertion, and collision resolution

A typical `get(key)` computes a hash, maps it to an index, then checks candidate entries. It first compares hash values and then key equality. Correct comparisons account for identical references and null according to implementation policy.

On `put(key, value)`, a matching key replaces its value and returns the old value. If no matching key exists, a new entry is linked or otherwise inserted into the bucket. Map size changes only for a new key. A `HashSet.add` similarly returns `false` when an equal element is already present.

Many hash tables use separate chaining: each bucket holds multiple entry nodes. Other hash tables use open addressing, but ordinary OpenJDK `HashMap` has traditionally used bucket chains with tree conversion under certain conditions.

Capacity, load factor, and resizing

A low load factor uses more buckets and generally shortens collision searches. A high load factor conserves table space but increases collisions. The default is intended as a general compromise, not a universal optimum.

When size crosses a threshold, a typical implementation allocates a larger bucket array and relocates or splits entries. The resize is `O(n)`, but geometric growth makes a long sequence of inserts expected amortized `O(1)` each. Pre-sizing can avoid repeated growth when an estimate is reliable. Excessive pre-sizing wastes memory and can slow full iteration because iteration may scan buckets as well as entries.

> **HotSpot note:** In current OpenJDK implementations, capacities are generally powers of two, index selection uses masked hash bits, and resizing often doubles capacity. Hash spreading mixes high bits into low bits. Details and helper methods can change by JDK release.

Tree bins

Long collision chains degrade lookup toward `O(n)`. Modern OpenJDK versions can transform a sufficiently populated bin into a red-black tree when the table is also large enough, giving `O(log n)` comparison depth for that bin under suitable conditions. Bins can later return to list form.

> **HotSpot note:** Treeification and untreeification thresholds, minimum table capacity, tie-breaking, and node layouts are version-sensitive. The Java API does not guarantee tree bins or worst-case logarithmic lookup. Treat them as resilience in a particular implementation, not as permission to use poor hashes.

Null, absence, and defaulting

`HashMap` permits one null key and multiple null values. Therefore `get(key) == null` can mean no mapping or a mapping to null. Use `containsKey` when the distinction matters. Prefer avoiding null values in domain maps when absence has a clear meaning.

`getOrDefault` returns the mapped value even if that value is null; it uses the default only when no mapping exists. `putIfAbsent` treats a null mapping as absent for its operation. Read each method's contract carefully.

Compute and merge methods

`computeIfAbsent` is ideal for multimap assembly:

JAVA

```
index.computeIfAbsent(customerId, ignored -> new ArrayList<>()).add(order);
```

The mapping function should be short and should not structurally mutate the same map recursively. If it returns null, no mapping is recorded. `computeIfPresent` runs only for a present non-null mapping. `compute` considers both presence and absence; a null result removes the mapping. `merge(k, incoming, remapper)` inserts `incoming` when absent or combines it with the current non-null value; a null remapping result removes the key.

These methods make a compound update concise on an ordinary map but do not make that map thread-safe. Concurrent map implementations define stronger per-key atomic behavior for their overrides.

Views and iteration

`keySet`, `values`, and `entrySet` are backed views. Iterate `entrySet` when both key and value are needed; repeated `map.get(key)` is redundant. Removing through a view removes mappings. A `HashMap` defines no stable iteration order. Even if a run appears consistent, a resize, JDK change, input change, or hash change can alter it.

Typical iterators are fail-fast under detected unsupported structural modification. Replacing the value for an existing key is often nonstructural, but this is not a general concurrency contract. `Map.Entry` objects from iteration may be tied to the map and should not be retained as permanent independent records; use `Map.entry(k, v)` or a domain record for a snapshot pair.

TRACE IT | Worked Java example

This frequency index uses an immutable composite key and `merge`:

JAVA

```java
import java.util.HashMap;
import java.util.List;
import java.util.Map;

record Endpoint(String method, String path) {
    public Endpoint {
        if (method == null || path == null) {
            throw new IllegalArgumentException("endpoint fields are required");
        }
        method = method.toUpperCase(java.util.Locale.ROOT);
    }
}

record Request(String method, String path) {}

final class RequestCounter {
    static Map<Endpoint, Long> count(List<Request> requests) {
        Map<Endpoint, Long> counts = new HashMap<>();
        for (Request request : requests) {
            Endpoint key = new Endpoint(request.method(), request.path());
            counts.merge(key, 1L, Long::sum);
        }
        return Map.copyOf(counts);
    }

    public static void main(String[] args) {
        var requests = List.of(
                new Request("get", "/health"),
                new Request("GET", "/orders"),
                new Request("GET", "/health"));
        System.out.println(count(requests));
    }
}
```

Normalization is part of key construction, so equality semantics match the domain rule that HTTP method case is irrelevant here. The path is intentionally not normalized; whether `/orders/` equals `/orders` is an application decision, not a hash-table concern.

`Map.copyOf` prevents result membership changes and rejects null keys or values. It does not promise the iteration order of the source hash map, so output formatting and tests must not depend on order.

TRACE IT | Execution or memory walkthrough

For the sample input:

1. `Endpoint("get", "/health")` becomes `("GET", "/health")`. Its hash selects a bucket. No equal key exists, so `merge` inserts count `1`.
2. `("GET", "/orders")` selects some bucket and is inserted with `1`.
3. A new `Endpoint("GET", "/health")` is a different reference but is record-equal to the first key and has the same hash. Lookup reaches the same bucket, `equals` succeeds, and `Long::sum` produces `2`.

Conceptually, with capacity eight:

TEXT

```
bucket 0: empty
bucket 1: [Endpoint(GET,/orders) -> 1]
bucket 2: empty
bucket 3: [Endpoint(GET,/health) -> 2]
bucket 4: empty
...
```

Actual indexes are deliberately omitted because hash spreading and table state are implementation-sensitive. The map stores one entry per unique endpoint, not one per request. Temporary equal key objects created for repeated endpoints become unreachable after lookup. The boxed `Long` values are replaced as counts grow because `Long` is immutable.

Now consider a broken mutable key:

JAVA

```java
final class MutableKey {
    String id;
    MutableKey(String id) { this.id = id; }
    public boolean equals(Object other) {
        return other instanceof MutableKey k && id.equals(k.id);
    }
    public int hashCode() { return id.hashCode(); }
}
```

After `map.put(key, value)`, changing `key.id` changes its lookup hash. `map.get(key)` can return null even though iteration still reveals the same reference. This violates the table's key-stability assumption, not the map's implementation.

Complexity and performance

For `n` entries and a reasonable hash distribution:

Operation	Expected	Collision-degraded	Notes
`get`, `put`, `remove`	`O(1)`	up to `O(n)` abstractly	tree bins may improve a particular implementation
`containsKey`	`O(1)`	up to `O(n)`	invokes hash/equality work
full iteration	`O(n + capacity)` typical	same form	oversized tables can hurt
resize	`O(n)`	`O(n)`	amortized across insertions
`HashSet` membership	same as backing hash table	same caveat	value is only a marker

Hash computation itself may not be constant in key size. `String.hashCode` is proportional to string length on first computation in many implementations, though caching and compact-string representation are implementation concerns. Expensive `equals` also increases collision cost.

Space is `O(n + capacity)` plus entry/node overhead. Load factor changes the trade-off between empty buckets and collision depth. `LinkedHashMap` adds ordering links. `IdentityHashMap` uses reference identity and a specialized representation. `EnumMap` uses enum ordinals and is typically compact. Choose semantics first, then measure.

WATCH OUT | Edge cases and common mistakes

- Overriding `equals` without a consistent `hashCode`.
- Mutating equality-relevant key state after insertion.
- Assuming unequal keys require different hash codes.
- Assuming iteration order or using printed `HashMap` order in golden tests.
- Using `get(k) == null` to prove absence when null values are permitted.
- Calling `containsValue` as though it were a hashed lookup; it generally scans values.
- Performing expensive or recursive side effects in `computeIfAbsent`.
- Assuming ordinary `HashMap.compute` is atomic across threads.
- Pre-sizing from an untrusted claimed count and allocating excessive memory.
- Using arrays as value keys without content-based wrappers; arrays inherit identity equality.
- Returning mutable lists stored as map values and calling the outer map immutable.
- Expecting tree bins to fix adversarial equality or expensive hash functions.
- Sharing a `HashMap` for concurrent mutation without synchronization.
- Forgetting that `HashSet` uniqueness is exactly the equality definition of its elements.

Production engineering notes

Prefer immutable, small, domain-specific keys. Normalize once at the boundary and make normalization rules explicit. Avoid concatenated string keys when components can be represented by a record; concatenation risks ambiguity and repeated allocation.

Size maps from credible estimates, accounting for load factor rather than setting initial capacity equal to expected entries and assuming no resize. Avoid massive speculative capacities. Monitor cache/index cardinality, eviction, and skew; an unbounded map is often a memory leak by design.

For user-controlled keys, use current supported JDKs and validate request sizes. Hash flooding can convert an expected constant-time endpoint into CPU exhaustion. Tree bins help in common OpenJDK versions but do not replace input limits, timeouts, and observability.

Use `LinkedHashMap` when encounter or access order is a contract, `TreeMap` for ordered range queries, `ConcurrentHashMap` for shared concurrent mutation, `EnumMap` for enum keys, and identity maps only when reference identity is truly the model. A synchronized wrapper still requires external locking around compound iteration or multi-call invariants.

When publishing maps, specify whether values are deeply immutable. `Map.copyOf` freezes mappings, not mutable objects reachable from values. Be cautious with cached negative or null results; explicit wrapper types often make state clearer.

TRY IT | Interview questions and model answers

How does `HashMap.get` work?

It computes a key hash, derives a candidate bucket, then checks entries in that bucket using hash and equality. Expected lookup is constant with good distribution; collisions require additional comparisons. Exact bucket and tree mechanics are implementation-specific.

Why must equal objects have equal hash codes?

The table uses the hash to choose where to search. If equal keys could select different regions, lookup could miss a logically identical stored key.

What happens when a key changes after insertion?

If equality or hashing changes, lookup searches using the new hash while the entry remains placed according to the old hash. Retrieval and removal can fail. Use immutable keys or stable identity fields.

What is a collision? Is it an error?

A collision is two unequal keys sharing a bucket or hash result. It is normal and must be resolved by equality checks. Excessive collisions hurt performance.

Why is `HashMap` lookup not simply guaranteed O(1)?

The API does not guarantee collision distribution, hash cost, or a particular bucket representation. Expected `O(1)` assumes suitable hashing and load. Worst-case abstract lookup can be linear.

How is `HashSet` commonly implemented?

It is commonly backed by a `HashMap`, storing set elements as keys and a shared private marker as the map value. That is an OpenJDK-style implementation fact, while the set contract only guarantees uniqueness.

When would you use `computeIfAbsent`?

For lazy per-key initialization such as grouping values into lists. The function should be short, return the desired initial value, and avoid structurally modifying the same map.

TRY IT | Exercises

1. Implement a correct immutable `Coordinate` key manually with `equals` and `hashCode`, then compare it with a record.
2. Dry-run three colliding keys through insert, replacement, lookup, and removal.
3. Write a program that demonstrates the mutable-key failure. Repair it by making the key immutable.
4. Build a word-frequency map using `merge`, then return entries sorted by frequency without relying on hash iteration order.
5. Given one million expected entries, explain how load factor, key size, entry overhead, and capacity affect memory. Identify what must be measured rather than guessed.
6. Compare an outer `Map.copyOf` containing mutable lists with a deeply unmodifiable result.

RECAP | Chapter summary

Hash tables narrow key search through a hash and confirm identity through `equals`. Their correctness depends on consistent equality and hashing plus stable keys. Collision handling, capacity, load factor, and resizing produce expected constant-time operations under ordinary assumptions, not an unconditional guarantee. OpenJDK's power-of-two tables and tree bins are useful implementation knowledge but are version-sensitive. Production design requires bounded cardinality, deliberate key semantics, explicit order and null policies, and a concurrency strategy.

TRY IT | Revision checklist

- [ ] I can state the `equals` and `hashCode` contract.
- [ ] I can derive lookup, insertion, collision handling, and resizing.
- [ ] I explain expected and amortized costs with assumptions.
- [ ] I know why mutable keys become unreachable by normal lookup.
- [ ] I distinguish absence from a null mapping.
- [ ] I can use `merge` and compute methods with their null semantics.
- [ ] I do not depend on `HashMap` or `HashSet` iteration order.
- [ ] I label tree bins, thresholds, hash spreading, and capacity shape as implementation details.
- [ ] I can select ordered, concurrent, enum, identity, or sorted alternatives by semantics.

SDE-2 CORE CHAPTER

Chapter 4 - SDE-2 Hashing, Maps, Sets, and Prefix-State Patterns

Why hashing is more than a lookup trick

Hashing turns a value or derived state into a key that can be found without scanning everything seen so far. That enables pair lookup, frequency counting, grouping, sequence starts, and prefix-state matching. In Java, however, algorithmic correctness depends on the object equality contract, key immutability, collision behavior, and memory cost.

An SDE-2 answer should never stop at "HashMap is O(1)." It should say expected or amortized time, describe the key and value meanings, prove why the stored state is sufficient, and explain what happens with duplicates, overflow, mutable keys, adversarial cardinality, and production concurrency.

Learning objectives

After completing this chapter, you should be able to:

- choose membership, frequency, value-to-index, grouping, or prefix-state hashing from problem signals;
- implement unsorted two-sum with a hash map and preserve the distinct-index contract;
- build immutable anagram signatures with correct `equals` and `hashCode`;
- derive longest-consecutive-sequence scanning from sequence starts;
- count target-sum subarrays from equalities between prefix sums;
- canonicalize prefix remainders to count divisible-sum subarrays with negative values;
- design immutable custom keys and explain why mutable keys become unreachable;
- state expected, amortized, and worst-case complexity honestly; and
- discuss capacity, cardinality, memory, concurrency, caching, and untrusted input.

CHOOSE IT | Recognition and decision map

Signal in the problem	Map or set state	Typical value stored
have I seen this value?	membership set	no separate value
how often has it appeared?	frequency map	count
which earlier index completes a relation?	value-to-index map	earliest or latest index
collect items with the same signature	grouping map	list or aggregate
does a sequence begin here?	membership set	no separate value
how many earlier prefixes have a required value?	prefix-state frequency map	number of occurrences
what was the earliest position of this state?	prefix-state index map	earliest index
cache result by multiple fields	composite key map	computed result/future

The stored value must match the question. Counting solutions requires frequencies; finding a longest range often requires the earliest index; returning any pair may need only one index. Using a set when multiplicity matters loses information.

Hash-map model and complexity language

A hash map computes a hash, chooses a bucket, and then uses equality to distinguish keys in that bucket. Equal keys must have equal hash codes. Unequal keys may share a hash code, so collision handling is part of normal operation, not an exceptional failure.

For a well-distributed hash function and controlled load factor, lookup, insertion, and removal are expected `O(1)`. Resizing occasionally costs `O(n)`, but a sequence of insertions is amortized expected `O(1)` per insertion. Worst-case behavior can be worse because collisions and key comparison still exist. Modern Java implementations contain collision defenses, but portable algorithm analysis should not depend on an undocumented bucket layout or claim a universal hard constant-time guarantee.

A map also has substantial memory overhead: table slots, nodes or internal entries, object headers, references, boxed primitive values, and unused capacity. `HashMap<Integer,Integer>` can use far more memory than two primitive arrays. For a small dense alphabet, an array is often the better data structure. For a sparse or large key space, hashing buys flexibility.

Java `HashMap` allows one null key and null values, while `ConcurrentHashMap` does not. Iteration order is not a stable API guarantee for `HashMap`. Use `LinkedHashMap` when insertion order is an explicit result requirement, or sort results before returning.

Equality, hash codes, and stable keys

The essential Java contracts are:

- `equals` is reflexive, symmetric, transitive, consistent, and false for null;
- if `a.equals(b)`, then `a.hashCode() == b.hashCode()`; and
- fields used by equality and hashing must not change while the object is a key.

The final rule is easy to violate. Suppose an `ArrayList<Integer>` is inserted as a key. Its equality and hash code depend on its elements. If an element changes, a later lookup computes a different bucket position, so even `map.get(theSameListReference)` may fail to find the entry through normal lookup. The entry has not vanished from memory; its key no longer leads to the bucket where it was stored.

Prefer records composed of immutable fields, immutable value objects, strings, enums, and primitive wrappers. A record containing an array is not deeply immutable: the generated `equals` compares that array by reference, and callers might mutate it. A custom array-backed key must make a defensive copy, use content equality, and never expose the array.

Cache a hash code only when every equality-relevant field is immutable. A cached hash with mutable contents makes the inconsistency even harder to diagnose.

Pattern 1: unsorted two-sum with a hash map

Given an unsorted array and target, scan from left to right. Before storing the current value, compute the complement and ask whether an earlier index produced it. If so, return the earlier and current indexes. Otherwise store the current value and its index.

The order of lookup and insertion enforces distinct indexes: the current element cannot match itself unless an equal value occurred earlier. The invariant before processing index `i` is that the map contains an index for each stored value in prefix `[0, i)`. If a solution ending at `i` exists, its complement is in that prefix and the lookup finds it.

TRACE IT | Dry run

For `[3, 2, 4, 3]` and target 6:

i/value	needed	prior map	action
0/3	3	`{}`	store `3 -> 0`
1/2	4	`{3=0}`	store `2 -> 1`
2/4	2	`{3=0,2=1}`	return `(1,2)`

The pair uses values 2 and 4. If the problem instead asks for every pair, storing one index is insufficient; duplicate policy and output-size complexity become central.

Compute `needed` in `long`. With `int` target and values, mathematical subtraction can leave the `int` range. A complement outside that range cannot appear in an `int[]`, so skip the lookup. The runnable method uses a real `HashMap`; it does not sort or silently replace this required pattern with two pointers.

Expected time is `O(n)`, space is `O(n)`. A sort-and-two-pointer alternative takes `O(n log n)` and may lose original indexes unless pairs are retained.

Pattern 2: frequency and membership

A frequency map summarizes a multiset. After processing prefix `[0, i)`, invariant `frequency[x]` equals the number of occurrences of `x` in that prefix. This supports duplicate detection, majority candidates with verification, intersection with multiplicity, and top-frequency selection.

Use `merge(key, 1, Integer::sum)` for concise counting when counts fit `int`. Use `long` counts for streams or aggregated datasets that can exceed that range. Removing zero counts keeps maps canonical in sliding-window algorithms and makes `map.size()` equal the number of active distinct keys.

A set answers membership only. For deduplication that must preserve encounter order, use `LinkedHashSet` or maintain a result list plus a `HashSet` of seen values. For sorted results, use sorting or `TreeSet`, accepting `O(log n)` operations.

Production code should cap distinct-key cardinality for untrusted or effectively unbounded input. A linear-time algorithm can still exhaust heap if every token is distinct.

Pattern 3: grouping anagrams by immutable signature

For lowercase ASCII words, the 26 letter counts form a canonical signature. Words are anagrams exactly when their signature vectors are equal. Map each signature to the list of words having it.

The custom `LowercaseSignature` in the runnable class owns a private count array, computes content-based equality with `Arrays.equals`, and uses `Arrays.hashCode`. Callers never receive the backing array. This repairs two common Java errors: using a raw array as a map key, which compares by identity, and reusing a mutable counting buffer across insertions.

TRACE IT | Dry run

Words `eat`, `tea`, `tan`, `ate`, `nat`, `bat` produce three logical keys:

```text
counts(a=1,e=1,t=1) -> [eat, tea, ate]
counts(a=1,n=1,t=1) -> [tan, nat]
counts(a=1,b=1,t=1) -> [bat]
```

A `LinkedHashMap` preserves first-group encounter order in the reference implementation, making tests deterministic.

Let `C` be the total number of characters and `g` the number of distinct signatures. Building groups takes expected `O(C)` time because the alphabet width is fixed, plus output storage. Signature storage costs `O(26g)`, abbreviated to `O(g)` for a fixed alphabet.

For Unicode text, first define normalization and case rules. Options include sorting code points, which costs `O(m log m)` per word, or building a sparse code-point frequency signature. Do not reuse the 26-letter solution without validating every code point.

Pattern 4: longest consecutive sequence

Insert every number into a set. A value begins a consecutive sequence only if its predecessor is absent. From each such start, walk upward while successors exist. Values inside a sequence are never used as starts, so each value participates in at most one forward walk.

Invariant during a walk: every integer from `start` through `current` is in the set, and the recorded length is exactly that range size. When the successor is absent, the sequence is maximal on the right; predecessor absence made it maximal on the left.

TRACE IT | Dry run

For `[100,4,200,1,3,2]`, the set contains all six values. `100` starts a length-1 sequence; `200` starts another. `4`, `3`, and `2` do not start because their predecessors exist. `1` has no predecessor, then walks through `2,3,4`, producing best length 4.

Expected time is `O(n)`, not `O(n^2)`: forward walks across all starts visit each distinct value once. Space is `O(n)`.

Handle numeric boundaries explicitly. Testing `value - 1` when `value == Integer.MIN_VALUE` wraps to `Integer.MAX_VALUE`; incrementing `Integer.MAX_VALUE` wraps to the minimum. The toolkit guards both operations so separate boundary values are not joined into a false cyclic sequence.

Prefix state: turn a subarray into two prefixes

Define half-open prefix sum:

TEXT

```
prefix[0] = 0
prefix[i + 1] = values[0] + ... + values[i]
```

Then the sum of subarray `[left, right)` is `prefix[right] - prefix[left]`. Hashing earlier prefix states lets the current right boundary find every left boundary satisfying a relation.

This is more general than a sliding window for arrays containing negative values. Sliding a left pointer based on sum is not monotone when adding or removing a negative value can move the sum in either direction. Prefix equalities remain valid.

The seed state for the empty prefix is essential. Store prefix sum zero with frequency one before scanning; this counts subarrays beginning at index zero.

Pattern 5: count subarrays with a target sum

At current prefix `P`, a prior prefix `Q` creates target sum `target` when:

```text
P - Q = target
Q = P - target
```

Therefore add the frequency of `P - target`, then increment the frequency of `P`. Query before insertion when zero-length subarrays are not part of the answer.

Invariant before each input value: the map counts every prefix ending before the current right boundary. `answer` counts every target-sum subarray whose right boundary has already been processed.

TRACE IT | Dry run

For `[1,2,1,2]`, target 3:

value	prefix P	seek P-3	prior count	answer	updated map entry
1	1	-2	0	0	`1 -> 1`
2	3	0	1	1	`3 -> 1`
1	4	1	1	2	`4 -> 1`
2	6	3	1	3	`6 -> 1`

The three subarrays are indexes `[0,2)`, `[1,3)`, and `[2,4)`. Use `long` for prefix and answer; the number of subarrays can reach `n * (n + 1) / 2`.

Expected time is `O(n)` and space is `O(n)`. If all inputs are nonnegative and only a longest or shortest range is needed, a window may use less state, but it is a different proof.

Pattern 6: count subarrays divisible by K

A subarray sum is divisible by positive divisor `k` when two prefixes have the same remainder modulo `k`:

```text
(P - Q) mod k = 0
P mod k = Q mod k
```

Maintain frequencies of prior canonical remainders. For each prefix, compute `Math.floorMod(prefix, divisor)` so negative sums map into `[0, k)`. Add the previous frequency of that remainder, then increment it. Seed remainder zero with frequency one.

TRACE IT | Dry run

For `[4,5,0,-2,-3,1]` and `k = 5`, canonical prefix remainders are `4,4,4,2,4,0`. Before updates their prior frequencies contribute `0,1,2,0,3,1`, totaling 7 divisible-sum subarrays.

Using Java `%` directly can produce negative remainders; prefixes that are congruent mathematically may then appear under different keys. Canonicalization fixes that. Reject divisor zero. The toolkit requires a positive divisor, avoiding the `abs(Long.MIN_VALUE)` trap.

When `k` is small and trusted, a `long[]` frequency table may be faster and more memory-efficient than a map. When `k` is huge or sparse, allocating an array of size `k` is unsafe, so a map is appropriate.

Earliest index versus frequency

The same prefix state can answer a different question by changing the map value.

- **Count subarrays:** store frequency of each state.
- **Longest subarray:** store the earliest index of each state and never overwrite it.
- **Shortest subarray under a direct equality:** store the latest index, if the proof supports it.
- **Existence:** store a set or return immediately.

For longest zero-sum subarray, when prefix `P` repeats at boundaries `j` and `i`, subarray `[j, i)` sums to zero. The earliest `j` gives the greatest length for a fixed `i`. Using a frequency map here would not provide the boundary needed for the range.

Runnable Java 21 reference implementation

The class below compiles as written. Run checkpoints with `java -ea HashingPatternToolkit`.

JAVA (continued 1/6)

```java
import java.util.ArrayList;
import java.util.Arrays;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedHashMap;
import java.util.List;
import java.util.Map;
import java.util.Set;

public final class HashingPatternToolkit {
    private HashingPatternToolkit() {
    }

    public record Point(int x, int y) {
    }

    public static final class LowercaseSignature {
        private final int[] counts;
        private final int hash;

        private LowercaseSignature(String word) {
            if (word == null) {
                throw new IllegalArgumentException("word must not be null");
            }
            this.counts = new int[26];
            for (int i = 0; i < word.length(); i++) {
                char current = word.charAt(i);
                if (current < 'a' || current > 'z') {
                    throw new IllegalArgumentException("expected lowercase ASCII word");
                }
                counts[current - 'a']++;
            }
```

JAVA (continued 2/6)

```
            this.hash = Arrays.hashCode(counts);
        }

        @Override
        public boolean equals(Object other) {
            return this == other
                    || other instanceof LowercaseSignature that
                    && Arrays.equals(this.counts, that.counts);
        }

        @Override
        public int hashCode() {
            return hash;
        }
    }

    public static int[] twoSumUnsorted(int[] values, int target) {
        requireArray(values);
        Map<Integer, Integer> indexByValue = new HashMap<>();
        for (int i = 0; i < values.length; i++) {
            long needed = (long) target - values[i];
            if (needed >= Integer.MIN_VALUE && needed <= Integer.MAX_VALUE) {
                Integer earlier = indexByValue.get((int) needed);
                if (earlier != null) {
                    return new int[] {earlier, i};
                }
            }
            indexByValue.putIfAbsent(values[i], i);
        }
        return new int[] {-1, -1};
    }
```

JAVA (continued 3/6)

```
    public static Map<Integer, Long> frequencies(int[] values) {
        requireArray(values);
        Map<Integer, Long> result = new HashMap<>();
        for (int value : values) {
            result.merge(value, 1L, Long::sum);
        }
        return result;
    }

    public static List<List<String>> groupLowercaseAnagrams(List<String> words) {
        if (words == null) {
            throw new IllegalArgumentException("words must not be null");
        }
        Map<LowercaseSignature, List<String>> groups = new LinkedHashMap<>();
        for (String word : words) {
            groups.computeIfAbsent(new LowercaseSignature(word), ignored -> new ArrayList<>())
                    .add(word);
        }
        List<List<String>> result = new ArrayList<>();
        for (List<String> group : groups.values()) {
            result.add(List.copyOf(group));
        }
        return List.copyOf(result);
    }

    public static int longestConsecutive(int[] values) {
        requireArray(values);
        Set<Integer> members = new HashSet<>();
        for (int value : values) {
            members.add(value);
        }
        int best = 0;
```

JAVA (continued 4/6)

```
        for (int value : members) {
            boolean hasPredecessor = value != Integer.MIN_VALUE
                    && members.contains(value - 1);
            if (hasPredecessor) {
                continue;
            }
            int length = 1;
            int current = value;
            while (current != Integer.MAX_VALUE && members.contains(current + 1)) {
                current++;
                length++;
            }
            best = Math.max(best, length);
        }
        return best;
    }

    public static long countSubarraysWithSum(int[] values, long target) {
        requireArray(values);
        Map<Long, Long> prefixFrequency = new HashMap<>();
        prefixFrequency.put(0L, 1L);
        long prefix = 0;
        long answer = 0;
        for (int value : values) {
            prefix += value;
            answer += prefixFrequency.getOrDefault(prefix - target, 0L);
            prefixFrequency.merge(prefix, 1L, Long::sum);
        }
        return answer;
    }

    public static long countSubarraysDivisibleBy(int[] values, long divisor) {
```

JAVA (continued 5/6)

```
        requireArray(values);
        if (divisor <= 0) {
            throw new IllegalArgumentException("divisor must be positive");
        }
        Map<Long, Long> remainderFrequency = new HashMap<>();
        remainderFrequency.put(0L, 1L);
        long prefix = 0;
        long answer = 0;
        for (int value : values) {
            prefix += value;
            long remainder = Math.floorMod(prefix, divisor);
            answer += remainderFrequency.getOrDefault(remainder, 0L);
            remainderFrequency.merge(remainder, 1L, Long::sum);
        }
        return answer;
    }

    public static Map<Point, Long> countPointVisits(List<Point> points) {
        if (points == null) {
            throw new IllegalArgumentException("points must not be null");
        }
        Map<Point, Long> visits = new HashMap<>();
        for (Point point : points) {
            if (point == null) {
                throw new IllegalArgumentException("points must not contain null");
            }
            visits.merge(point, 1L, Long::sum);
        }
        return Map.copyOf(visits);
    }

    private static void requireArray(int[] values) {
```

JAVA (continued 6/6)

```
        if (values == null) {
            throw new IllegalArgumentException("values must not be null");
        }
    }

    public static void main(String[] args) {
        assert Arrays.equals(twoSumUnsorted(new int[] {3, 2, 4, 3}, 6),
                new int[] {1, 2});
        assert Arrays.equals(twoSumUnsorted(new int[] {3, 3}, 6),
                new int[] {0, 1});
        assert frequencies(new int[] {4, 4, 2}).equals(Map.of(4, 2L, 2, 1L));

        List<List<String>> groups = groupLowercaseAnagrams(
                List.of("eat", "tea", "tan", "ate", "nat", "bat"));
        assert groups.equals(List.of(
                List.of("eat", "tea", "ate"),
                List.of("tan", "nat"),
                List.of("bat")));

        assert longestConsecutive(new int[] {100, 4, 200, 1, 3, 2}) == 4;
        assert longestConsecutive(new int[] {Integer.MIN_VALUE, Integer.MAX_VALUE}) == 1;
        assert countSubarraysWithSum(new int[] {1, 2, 1, 2}, 3) == 3;
        assert countSubarraysWithSum(new int[] {0, 0, 0}, 0) == 6;
        assert countSubarraysDivisibleBy(new int[] {4, 5, 0, -2, -3, 1}, 5) == 7;

        Map<Point, Long> visits = countPointVisits(
                List.of(new Point(1, 2), new Point(1, 2), new Point(2, 1)));
        assert visits.get(new Point(1, 2)) == 2L;
        assert visits.get(new Point(2, 1)) == 1L;
    }
}
```

Complexity and state table

Pattern	Time	Space	Map meaning
unsorted two-sum	expected `O(n)`	`O(n)`	value to earlier index
frequency counting	expected `O(n)`	`O(distinct)`	value to count
lowercase anagram grouping	expected `O(C)`	`O(C + groups)` including output	signature to words
longest consecutive	expected `O(n)`	`O(n)`	membership only
target-sum subarray count	expected `O(n)`	`O(n)`	prefix sum to frequency
divisible-sum subarray count	expected `O(n)`	`O(min(n,k))` states	remainder to frequency

These are expected hash-table bounds. Returned groups and maps are output space; do not hide them inside or outside auxiliary-space notation inconsistently.

WATCH OUT | Edge cases and common mistakes

1. **Sorting in a required hash solution.** It changes the demonstrated pattern and may lose original indexes.
2. **Inserting before complement lookup.** The same element can match itself when target is twice its value.
3. **Subtraction overflow.** Compute complements and prefix states in `long`.
4. **Wrong map value.** A count problem needs frequencies; a longest-range problem often needs earliest indexes.
5. **Missing empty-prefix seed.** Subarrays starting at zero disappear.
6. **Query after update.** For nonempty subarrays, count prior prefixes before inserting the current one.
7. **Negative remainder split.** Use `floorMod` or another canonical remainder formula.
8. **Boundary wraparound in sequences.** `MIN_VALUE - 1` and `MAX_VALUE + 1` wrap.
9. **Array as key.** Java arrays use identity equality and identity-based hash codes unless wrapped in a content-based key.
10. **Mutable key.** Changing equality-relevant fields after insertion breaks lookup.
11. **Hash code without equality.** Overriding only one side of the contract yields incorrect behavior.
12. **Assuming iteration order.** `HashMap` order is not a result contract.
13. **Unbounded cardinality.** Expected linear time does not prevent memory exhaustion.
14. **Concurrent unsynchronized mutation.** `HashMap` is not a thread-safe shared mutable cache.
15. **Hash as proof of equality.** A hash collision must be resolved with `equals` or exact verification.

SDE-2 production follow-ups

- **Capacity planning:** if a trustworthy size estimate exists, pre-size a map to reduce resizing, but do not allocate from an unbounded attacker-provided count. Measure because capacity formulas and implementation details can change.
- **Primitive specialization:** high-volume integer maps may justify primitive collections to avoid boxing. Introduce a dependency only after profiling and with operational familiarity.
- **Concurrency:** use method-local maps for algorithms. Shared caches need `ConcurrentHashMap` or an external cache plus atomic loading, eviction, expiry, and failure policy.
- **Cache semantics:** a hash map is not a complete cache. Bound entries, account for value weight, avoid caching failures forever, and prevent stampedes for expensive keys.
- **Skew and hot keys:** one group or frequency key may dominate memory or contention. Track maximum group size and distinct cardinality, not only total input.
- **Untrusted keys:** cap lengths and counts, use stable library key types, and prefer deterministic algorithms where collision-sensitive work can be abused.
- **Persistence:** Java hash codes are not durable cross-language identifiers. Persist canonical fields, not in-memory bucket hashes.
- **Data privacy:** map keys can contain customer identifiers or text. Avoid raw-key logging and sanitize metrics to prevent high-cardinality telemetry.
- **Result determinism:** sort keys or use a defined ordered map when API consumers, snapshots, or tests require stable ordering.
- **Null policy:** reject nulls at the boundary or specify them. Mixing "absent" with "present and mapped to null" complicates `get`-based logic.
- **Equality evolution:** adding an equality field to a key changes grouping and cache identity. Treat key schema as an API and migration concern.

TRY IT | Exercises with model checkpoints

Exercise 1: all unique two-sum value pairs

Return every distinct value pair summing to target from an unsorted array.

Model checkpoints: distinguish value pairs from index pairs; use a seen set and a canonical pair key to avoid duplicates; widen complement arithmetic; output can be `O(n)`; define ordering for deterministic results.

Exercise 2: longest zero-sum subarray

Return the boundaries of the longest contiguous range summing to zero.

Model checkpoints: map each prefix sum to its earliest boundary index; seed `0 -> 0` when prefixes are indexed by element count; on repetition at boundary `right`, candidate is `[earliest, right)`; never overwrite earliest; use `long` prefix.

Exercise 3: exactly K distinct subarrays

Count subarrays with exactly `k` distinct values.

Model checkpoints: derive `exactly(k) = atMost(k) − atMost(k − 1)`; each at-most helper uses a frequency map and monotone window; define `k <= 0`; use `long` result; remove keys at zero.

Exercise 4: top K frequent values

Return the `k` most frequent integers.

Model checkpoints: frequency map plus size-`k` min-heap gives expected `O(n + d log k)` for `d` distinct values; define tie order; validate `k`; bucket frequency can be linear but allocates by input length; output order is a contract.

Exercise 5: subarrays with equal zeroes and ones

Count ranges containing equal numbers of zero and one.

Model checkpoints: transform zero to -1 and one to +1; equal counts become zero-sum; reuse prefix-frequency proof; reject other input values or define them; seed empty prefix.

Exercise 6: Unicode anagram key

Design an immutable signature for normalized Unicode strings.

Model checkpoints: choose normalization and case policy; sorted code points give an immutable sequence but cost `O(m log m)`; sparse counts need a stable canonical representation; copy mutable inputs; test canonically equivalent spellings and supplementary code points.

Exercise 7: bounded frequency service

Count events by key in a long-running service with a strict memory budget.

Model checkpoints: an exact unbounded map cannot satisfy the budget for unbounded keys; define time windows, eviction, aggregation, or approximate sketches; identify heavy hitters; specify concurrency and persistence; expose dropped/evicted-state metrics.

Exercise 8: custom composite key review

Review a key class containing `String tenant`, `byte[] digest`, and mutable `List<String> tags`.

Model checkpoints: copy the byte array and tags into immutable representations; define order sensitivity; implement equality and hash from the same fields; validate nulls; avoid exposing internals; consider whether every field truly belongs to identity.

Interview answer checklist

- [ ] I named exactly what the map key and value represent.
- [ ] I used a set only when multiplicity and indexes are unnecessary.
- [ ] I stated expected/amortized complexity rather than promising universal `O(1)`.
- [ ] I proved distinct-index behavior in two-sum.
- [ ] I seeded the empty prefix when ranges may begin at zero.
- [ ] I chose frequency versus earliest index based on the requested output.
- [ ] I canonicalized negative remainders.
- [ ] I guarded integer boundary wraparound.
- [ ] My custom keys are immutable and implement consistent equality and hashing.
- [ ] I defined result ordering, null policy, and duplicate behavior.
- [ ] I can discuss cardinality, memory, concurrency, and hostile input.

RECAP | Summary

Hashing is a way to retain exactly the past state a future element needs. Unsorted two-sum maps values to earlier indexes. Frequency maps preserve multiplicity; grouping maps canonical signatures to outputs; a set lets longest-consecutive scanning begin only at maximal sequence starts. Prefix-state hashing turns a subarray equation into a lookup for an earlier sum or remainder. All of these rely on stable equality, collision resolution, correct numeric width, and the right map value. The SDE-2 standard includes not only the expected linear algorithm, but also immutable keys, honest complexity, bounded cardinality, deterministic API behavior, and production ownership and concurrency decisions.

Part II - Practice and Handoff

Practice Ladder and Next Steps

TRY IT | Practice ladder

- Foundation: frequency and deduplication
- Interview Core: pair sums, grouping, and prefix-state counts
- SDE-2 Follow-up: custom keys, capacity, and memory
- Challenge: combine rolling state with hash lookup

For every coding problem, state the input contract, select the numeric and collection types deliberately, write the invariant before the loop or recursion, test the smallest and largest valid inputs, and close with time and auxiliary-space complexity.

Navigation

Previous book: Study Step 07 - Strings and String Problem-Solving Patterns

Series index

Next book: Study Step 09 - Recursion and Backtracking in Java

TRY IT | Completion check

- [ ] I can recognize the core patterns without being told the data structure or algorithm.
- [ ] I can implement the representative Java templates from memory.
- [ ] I can explain why each implementation is correct.
- [ ] I can test boundaries, invalid inputs, overflow, and adversarial shapes.
- [ ] I can answer at least one SDE-2 follow-up involving trade-offs or production constraints.

Series Roadmap

Complete the study steps in order when rebuilding from fundamentals, or enter at the first step whose readiness checks expose a gap. The same public study codes appear on the website and every PDF cover. Click a title when your PDF viewer permits local sibling-file links.

STEP	FOCUSED LEARNING PATH
01	Java Foundations for Problem Solving
02	Time and Space Complexity for Java Interviews
03A	Number Systems and Math Foundations for DSA Interviews
03B	Number Systems Interview Patterns and Rapid Revision
04	Bit Manipulation in Java
05	Loop Mastery, Patterns, and Index Calculations
06	Arrays and Array Problem-Solving Patterns
07	Strings and String Problem-Solving Patterns
08	Hashing: Maps, Sets, Frequency, and Prefix State
09	Recursion and Backtracking in Java
10	Linked Lists: Pointer Reasoning and Mutation
11	Stacks, Queues, Deques, and Monotonic Patterns
12	Binary Search: Bounds, Answers, and Invariants
13	Trees, Binary Search Trees, and Tries
14	Heaps, Priority Queues, Selection, and Top-K
15	Graphs: Traversal, Ordering, Paths, and Union-Find
16	Greedy Algorithms: Recognition, Proofs, and Scheduling
17	Dynamic Programming: State, Transitions, and Optimization
18A	Advanced Java A: JVM and Execution
18B	Advanced Java B: Language, OOP, and Modern Java
18C	Advanced Java C: Collections, Streams, and I/O
18D	Advanced Java D: Concurrency and the Memory Model
18E	Advanced Java E: Performance, Diagnostics, and GC Incidents
18F	Advanced Java F: Design, Backend, Testing, and Security
18G	Advanced Java G: Question Bank, Study Plan, and Reference
18H	Advanced Java H: Spring Boot and REST APIs
18I	Advanced Java I: Persistence, SQL, JPA, and Caching
18J	Advanced Java J: Distributed Systems and System Design

Study Step 03 uses linked foundations (03A) and workbook (03B) books. Study Step 18 uses ten focused books (18A-18J), including a three-book backend specialist track. These suffixes keep every file focused while preserving one unambiguous route.

About the Author

Vinay Reddy Kalluri

Vinay Reddy Kalluri is a senior Java backend engineer, the creator and founding author of the Java SDE-2 Interview Preparation Series, and its Editor-in-Chief and Chief Auditor. His work spans high-throughput microservices, event-driven architecture, resilient APIs, large-scale data processing, cloud delivery, and production performance across healthcare and enterprise systems.

Editorial leadership

As Editor-in-Chief, Vinay owns the learning sequence, scope, voice, and publication decisions. As Chief Auditor, he owns the Java accuracy standard, validation evidence, attribution review, and release-readiness gate. Individual contributors retain visible credit for accepted original work through AUTHORS.md and the public Git history.

What distinguishes his approach is the combination of hands-on system design and operational ownership. He treats timeouts, retries, idempotency, dead-letter recovery, data consistency, SQL behavior, caching, and observability as part of the design rather than afterthoughts. He has also mentored engineers and led modernization, migration, and reliability work across distributed services.

Selected engineering and academic highlights

- More than eight years building Java and Spring Boot services for high-throughput healthcare and enterprise platforms.
- Designed Kafka-based event systems that have processed more than one million events per hour, with explicit idempotency, retry, recovery, and observability controls.
- M.S. in Computer Science from the University of Missouri-Kansas City (3.9/4.0) and M.S. in Software Engineering from VIT University (9.3/10), where he was a Gold Medalist.
- Independent builder of Work Visa Insights, an immigration-data intelligence platform, and Play Together, a published Android application.

Why this series exists

Vinay created this series to turn fragmented interview preparation into a deliberate progression: learn the fundamentals, run the examples, predict behavior, debug failures, explain trade-offs, and only then move into advanced engineering. The books reflect the same principle he applies to production systems: clarity before cleverness, explicit contracts, measurable behavior, and reliable foundations.

Connect: LinkedIn | GitHub

Copyright and Publishing Notes

Examples target Java 21 unless a section states otherwise. Validate release-specific APIs, JVM behavior, security, performance, licensing, and operational requirements before production use.

Series position: Study Step 08 of 18. The local table of contents and PDF bookmarks navigate within this file; the roadmap and printed filenames navigate across the complete series.

Source provenance: curated from the independent Java SDE-2 master guide plus focused original material. Original master chapter numbers are labeled explicitly when retained in cross-references.